

Solution Design Document: SmartAQI

AI-Powered Air Quality Monitoring and Decision Support System for Indian Cities

Prepared by: Vaibhav Arya (Lead Architect) & AI Assistant (Co-Designer)

Target: 100% Free (₹0 Cost) Prototype Implementation & Simulation

Document State: Final Design (Ready for Development)

Date: June 2026

1. Problem Definition

Rapid urbanization in India has led to severe air pollution, particularly in northern and industrial hubs like Delhi, Patna, and Ahmedabad. Analysis of historical ambient air quality data (2015–2020) provides crucial local insights that define the scope of our proposed system:

- Dominant Pollutants:** Carbon Monoxide (CO) shows the strongest positive correlation with AQI ($r = 0.65$), followed closely by Fine Particulate Matter (PM2.5) ($r = 0.63$). This indicates that our prototype must prioritize tracking combustion-related gases and fine dust particles.
- Telemetry Anomalies:** The historical dataset reveals extreme outliers—such as Ahmedabad recording an astronomical AQI peak of 2049.0 on February 19, 2018 (with CO levels at 132.47 mg/m³), and another massive CO spike of 175.81 mg/m³ in October 2017. These readings represent either localized pollution events (e.g., waste fires) or monitoring equipment malfunctions.

Solving this problem is critical. Long-term exposure to these levels of CO and PM2.5 leads to severe respiratory illness. Our project aims to build a functional, simulated local network that detects these anomalies, alerts citizens, and triggers municipal responses.

2. Existing System & Gap Analysis

How Air Quality is Currently Handled

Air quality monitoring in Indian cities is run by the Central and State Pollution Control Boards (CPCB/SPCB) using Continuous Ambient Air Quality Monitoring Stations (CAAQMS). While highly accurate, these stations are expensive (costing upwards of ₹1.5–2 million each) and sparsely distributed.

Inefficiencies and Gaps in the Current System

- Sparse Spatial Coverage:** A major city might have only 3 to 5 official stations, leaving massive blind spots. Air quality can vary drastically between a heavy traffic intersection, a park, and a residential area.
- No Real-Time Validation:** Telemetry is published raw. Faulty sensor readings (such as the 2049.0 AQI anomaly in the dataset) are broadcast without automated validation, leading to public confusion or false alarms.
- Lack of Localized Predictions:** Existing systems only report what *has happened*. There is no accessible, local mechanism to forecast air quality drops for the next 24 hours.
- Disconnected Actions:** Official alerts are purely advisory. There is no automated connection between a pollution spike and localized municipal mitigation efforts.

[!NOTE]

Our Focus: Since deploying physical hardware nodes incurs costs, we will build a **100% free (₹0) virtual system**. We will create a Python simulator to model a multi-station sensor grid, and route this

data through open-source databases and Python-based machine learning to validate data and trigger alerts.

3. Proposed Smart System: SmartAQI

Objective

To build a modular, 100% free prototype of an air quality monitoring and automated decision-support system. It will ingest simulated multi-node sensor telemetry, filter anomalies, forecast local AQI using machine learning, and dispatch free automated alerts.

System Working (Prototype Workflow)

- 1. Virtual Data Generation:** A Python-based IoT simulator script runs continuously, generating PM2.5, PM10, and CO readings for 10 distinct coordinates across a mock city map, simulating diurnal traffic spikes and weather trends.
- 2. Ingestion & Validation:** The simulator transmits readings via HTTP POST to our backend server. A validation script cleans the data, comparing readings to historical limits and checking spatial neighbors. If a node generates an impossible spike (like CO > 100 mg/m³ while its neighbors read 1.2 mg/m³), the system flags it as an anomaly.
- 3. Forecasting Engine:** Validated hourly averages are processed by a scikit-learn model to forecast the next 24-hour AQI based on trends and weather parameters fetched from a free meteorological API.
- 4. Alert Dispatch:** If the forecast exceeds the "Poor" threshold, our backend automatically sends an alert to a Telegram channel and triggers a mitigation ticket on our web dashboard.

Major System Components

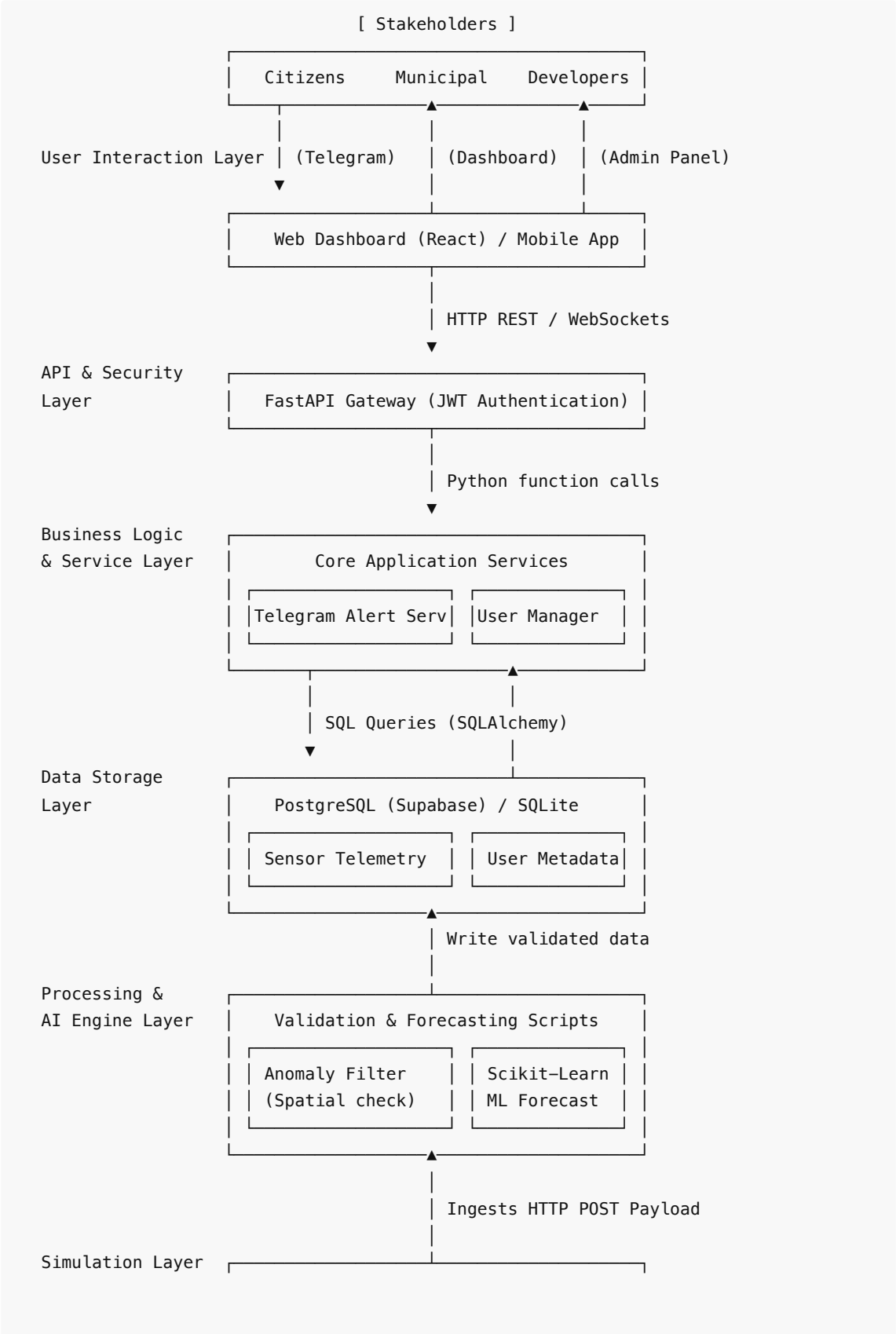
- **Python Telemetry Simulator (`simulator.py`):** Generates synthetic air quality data with built-in daily fluctuations, noise, and programmed telemetry anomalies for multi-node simulation.
- **FastAPI Backend Application:** A lightweight Python web server that handles data ingestion endpoints, validation logic, and the ML pipeline.
- **PostgreSQL / SQLite Database:** Relational database setup to store sensor feeds, validated records, and user settings. We will use SQLite for local development and Supabase's free tier for cloud deployment.
- **Machine Learning Engine:** Scikit-learn scripts for data validation (Random Forest anomaly detection) and forecasting (Linear Regression/XGBoost).
- **React Web Dashboard:** A simple frontend utilizing Leaflet.js (a free mapping library) and Chart.js to display real-time AQI maps and trend graphs.
- **Notification Routing Service:** A lightweight integration using the free Telegram Bot API and SMTP to dispatch instant alerts.

Stakeholders

- **Vulnerable Citizens:** Receive instant local health warnings on Telegram.
 - **Municipal Operators:** View the dashboard to identify hotspots and deploy simulated localized measures.
 - **Developers / Researchers (Us):** Able to audit the system, clean historical logs, and refine the forecasting models.
-

4. System Architecture

This architecture is structured around tools we can write and deploy locally (using Python, Node.js, and Docker) and host on free cloud tiers.



(100% Free)

Python Multi-Node Simulator

Component Details

- **Python Multi-Node Simulator:** Runs locally as an independent process. It models 10 virtual sensors across a map and pushes telemetry payloads over HTTP.
- **FastAPI Backend:** Coordinates the application, exposes endpoints for the sensor payload, and hosts token-based (JWT) security for the dashboard.
- **PostgreSQL / SQLite Database:** SQLite is used for development, migrating to local PostgreSQL for production. It uses SQLAlchemy ORM to manage sensor logs and application data.
- **Validation & ML Engine:** Runs within the FastAPI process. The anomaly filter checks if the input is within physiological limits and runs a spatial-consistency algorithm to flag miscalibrated sensors. A trained scikit-learn model forecasts upcoming AQI using historical data.
- **Telegram Alert Service:** A free service that invokes a simple HTTP request (<https://api.telegram.org/bot<token>/sendMessage>) to broadcast alerts to public channels.
- **React Web Dashboard:** Displays a color-coded map using Leaflet.js and plots AQI trends over time with Chart.js.

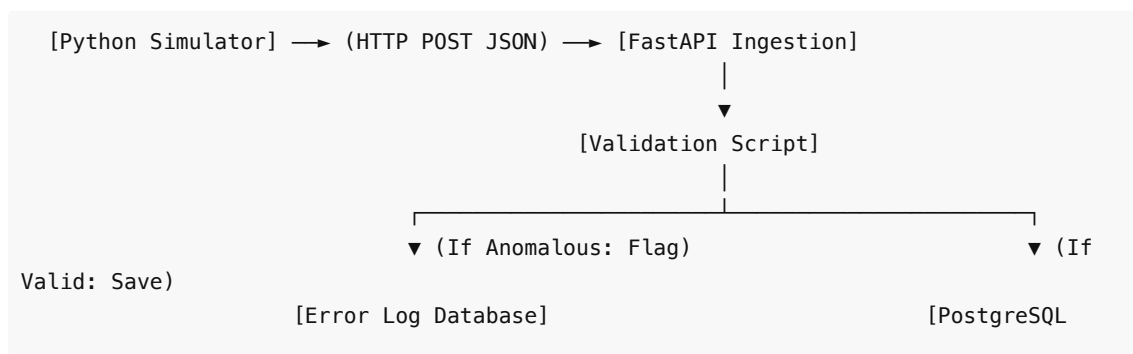
5. Data Flow

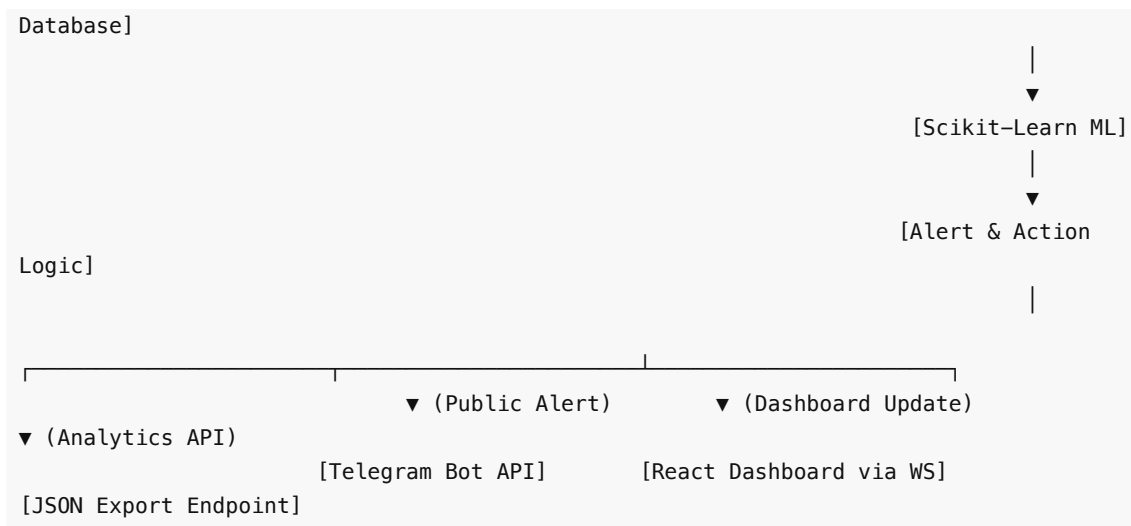
The process of capturing, validating, storing, predicting, and acting upon air quality data.

Step-by-Step Data Flow

1. **Data Generation:** Every 5 minutes, the Python simulator runs, generates pollutant values (PM2.5, PM10, CO) for the mock city coordinates, and transmits the JSON payload.
2. **Ingestion & Sanity Filtering:** The FastAPI endpoint receives the payload. A Pydantic schema validates the structure. A quick sanity check ensures values aren't negative or empty.
3. **Spatial Anomaly Detection:** The validation module queries the database for active nodes within a 2 km radius. If the node's reading deviates by more than 3 standard deviations from its neighbors, it is flagged as `Anomalous` and quarantined.
4. **Database Write:** Clean, validated logs are written to the database.
5. **AQI Forecasting:** Once per hour, a background task triggers the Scikit-learn model. It predicts the average AQI for the next day, factoring in wind speed and humidity variables retrieved from a free meteorological API.
6. **Notification Trigger:** If the forecasted AQI is above 150, the backend calls the Telegram Bot API to notify users. The dashboard is updated in real time via WebSockets.

Data Flow Diagram





6. Conceptual Technologies

These technologies are fully open-source, free to use, and can be developed on a local workstation:

- **Python Node Simulator:** Multi-node mock script using `numpy` and `requests` to generate realistic sensor streams and edge failures.
- **FastAPI (Python Backend):** High performance, automatic Swagger documentation generation, and easy asynchronous task scheduling.
- **PostgreSQL (Supabase Free Tier):** A free hosted cloud PostgreSQL instance that provides a secure, remote datastore with no registration costs.
- **Scikit-Learn & XGBoost (Python):** Lightweight libraries for linear regression, random forests, and gradient boosting. They consume very little RAM, making them ideal for running alongside our web server.
- **Leaflet.js & Chart.js (Frontend):** Leaflet is a lightweight mapping tool that works with open-source map tiles, avoiding Mapbox/Google Maps licensing costs. Chart.js is perfect for rendering responsive line graphs.
- **Telegram Bot API:** Completely free messaging gateway. Requires no credit card or business registration, making it ideal for prototype notifications.

7. Expected Impact

- **Environmental Impact:** Pinpoints local trash burning or heavy congestion zones, allowing local resident groups or municipal staff to take immediate, targeted action.
- **Health & Safety:** Enables asthmatics and outdoor workers to check micro-local conditions and receive notification alerts prior to severe pollution buildup.
- **Academic/Development Feasibility:** Provides a complete end-to-end framework that can be built, coded, and demonstrated in a college or internship setting.
- **Scalability:** The code can easily scale from running on a local laptop to a cloud VM (e.g., free tier on Render or Vercel) to monitor a larger municipal pilot zone.

8. Challenges & Limitations

Synthetic vs. Real Telemetry

Simulated data can sometimes be too perfect and fail to reflect the chaotic nature of physical environments.

- **Our Solution:** Inject noise, temporary connection dropouts, and multi-sensor correlation fluctuations directly into the `simulator.py` script to simulate realistic urban environments.

Free Cloud Tier Sleep Cycles

Free hosting platforms (like Render) spin down web servers after 15 minutes of inactivity, causing initial latency spikes.

- **Our Solution:** Configure the Python simulator to send telemetry pings every 5 minutes. This constant traffic keeps the free container awake and operational.
-

9. Future Scope

- **Physical Hardware Integration:** The modular API design allows us to drop in physical ESP32 nodes and PMS7003 sensors seamlessly by replacing the simulator endpoints.
- **Transition to Edge AI:** Implementing TensorFlow Lite for Microcontrollers (TinyML) directly on ESP32 boards if hardware is introduced.
- **Public Web Portal:** Building a fully public, interactive map showing real-time street-level air quality throughout the municipal pilot zone.
- **Weather API Integration:** Enhancing the forecasting algorithm by dynamically pulling temperature, wind direction, and rain forecasts from open APIs.